

Concurrent and Distributed Systems

Lab 3 – Java Synchronisation Utilities

Tasks

- Read the slides **Java Synchronisation Utilities**
- Go through the following examples and complete the exercises. The source code from the slides and the solutions are in **cds.lab3.zip**

Examples

Bike rental

Run the bike rental example from Lab 3, slides 8-12. Is the printed number of available bikes correct?

Print queue

Implement the first example from Java 9 Concurrency Cookbook, Chapter 3 Thread Synchronization Utilities: Controlling concurrent access to multiple copies of a resource.

IllegalMonitorStateException

In Lab 3, slides 22-26, study the example generating an **IllegalMonitorStateException**, so you can avoid this mistake in your future implementations. Read the explanations, implement the examples and run the code.

Exercises

Car park scenario

Provide a complete implementation of the car park scenario (Lecture 3, slides 23-26) using a monitor.

- The monitor class (**CarParkControl**) is provided in the slides
- Your task is to **implement 2 Java classes**
 - one for **arrivals**
 - one for **departures**(either by extending **Thread** or by implementing the **Runnable** interface)
- Use **multiple threads** to simulate cars **arriving at** or **leaving** the car park
- Print **console messages** to show:
 - car arrivals and departures
 - the current number of free parking spaces
- Introduce **random delays** to simulate arrival and departure times
- **Optional challenge:** go beyond the basics! Add a simple animation or visualisation of the car park
- Share your solution with us and the other students during the lab session!
- Compare this car park scenario with the one from Chapter 2 of Java 9 Concurrency Cookbook - Recipe 1. What are the similarities and the differences?

Grocery shop

At a grocery shop, customers can use 4 self-checkout tills. Each customer has a random number of products in their baskets, so their scanning time varies. Customers wait in a FIFO queue to access one of the 4 tills. Implement this scenario with an arbitrary number of customers while ensuring fair access to the tills.

Photo booth

A photo booth has space only for one customer and offers 4 operations:

1. passport – digital photos
2. passport – paper photos
3. funny – digital photos
4. funny – printed photos

A number of customers wander around the shopping centre. They approach the photo booth and check the curtain:

- If someone is in the booth, they leave, wander around for a few more minutes and try again.
- If the booth is free, the customer enters, chooses one of the 4 operations (possibly more than one) and then leaves.

Key difference from grocery shop: There is **no queue and no fairness**. Any wandering customer may enter the booth when it becomes free - no priority is given to those who have been waiting the longest.

Resources

- J. Fernandez Gonzalez (2017), **Java 9 Concurrency Cookbook** (2nd edition), Packt Publishing. You can access the online version through the library (log in with your university account), link [here](#). The GitHub repository providing the complete Java classes for each book chapter example is here: <https://github.com/PacktPublishing/Java-9-Concurrency-Cookbook-Second-Edition>
- <https://www.baeldung.com/java-illegalmonitorstateexception>
- <https://examples.javacodegeeks.com/java-basics/exceptions/java-lang-illegalmonitorstateexception-how-to-solve-illegalmonitorstateexception/>